

Code Explanation

Delta Live Tables Pipeline Explanation

The provided code is a Delta Live Tables (DLT) pipeline written in Python, utilizing Apache Spark for data processing. It ingests customer and order data from CSV files, cleans and transforms the data, and creates several tables for analysis.

Overview

This DLT pipeline is designed to process customer and order data, creating multiple tables with varying levels of data quality and aggregation. The pipeline consists of several stages, including data ingestion, cleaning, transformation, and aggregation.

Step 1: Define Source Tables

This step defines the source tables for customer and order data. The `customer\_bronze` and `order\_bronze` tables are created by reading CSV files from a specified location.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data loaded from CSV"
)
def customer_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customers_data")
    )
```

Step 2: Clean Customer Data

This step cleans the customer data by removing nulls and duplicates. The `customer\_silver` table is created with the cleaned data.

```
@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with nulls and duplicates removed"
)
def customer_silver():
    return (
        dlt.read("customer_bronze")
        .dropDuplicates(["CustId"])
        .filter(
            (F.col("CustId").isNull()) &
            (F.col("Name").isNull()) &
            (F.col("EmailId").isNull()) &
            (F.col("Region").isNull())
        )
    )
```

Step 3: Clean Order Data

This step cleans the order data by removing nulls and duplicates, and adds a `TotalAmount` column. The `order\_silver` table is created with the cleaned data.

```
@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return (
        dlt.read("order_bronze")
        .dropDuplicates(["OrderId"])
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
    )
```

Step 4: Create SCD Type 2 ordersummary Table

This step creates the `ordersummary` table, which is a Slowly Changing Dimension (SCD) Type 2 table that combines customer and order data. The table is partitioned by the `Date` column and has several constraints to ensure data quality.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table combining customer and order data",
    table_properties={
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    },
    partition_cols=["Date"],
    spark_conf={
        "spark.databricks.delta.schema.autoMerge.enabled": "true"
    }
)
@dlt.expect_all_or_drop({
    "valid_custid": "CustId IS NOT NULL",
    "valid_orderid": "OrderId IS NOT NULL",
    "valid_date": "Date IS NOT NULL"
})
def ordersummary():
    # implementation details
```

Step 5: Implement SCD Type 2 Logic

This step implements the SCD Type 2 logic for the `ordersummary` table. It checks for changes in customer data and updates the table accordingly.

```

try:
    # Check if the target table exists
    spark.sql("SELECT 1 FROM gen_ai_poc_databrickscoe.sdmc_wizard.ordersummary LIMIT 1")

    # Get current customer data
    current_customer_data = dlt.read("customer_silver")

    # Get existing customer data from ordersummary
    existing_customer_data = (
        spark.table("gen_ai_poc_databrickscoe.sdmc_wizard.ordersummary")
        .filter(F.col("IsActive") == True)
        .select("CustId", "Name", "EmailId", "Region")
        .distinct()
    )

    # Find changed customers
    changed_customers = (
        current_customer_data
        .join(
            existing_customer_data,
            "CustId",
            "inner"
        )
        .filter(
            (F.col("current_customer_data.Name") != F.col("existing_customer_data.Name")) |
            (F.col("current_customer_data.EmailId") != F.col("existing_customer_data.EmailId")) |
            (F.col("current_customer_data.Region") != F.col("existing_customer_data.Region"))
        )
        .select("CustId")
    )

    # Expire old records
    update_expr = {
        "IsActive": "false",
        "EndDate": f"'{current_timestamp}'"
    }

    condition = "t.CustId = s.CustId AND t.IsActive = true"

    delta_table = DeltaTable.forName(spark, "gen_ai_poc_databrickscoe.sdmc_wizard.ordersummary")
    delta_table.alias("t").merge(
        changed_customers.alias("s"),
        condition
    ).whenMatchedUpdate(
        set=update_expr
    ).execute()

    # Join current customer and order data
    new_records = (
        dlt.read("customer_silver")
        .join(

```

```

        dlt.read("order_silver"),
        "CustId",
        "inner"
    )
    .select(
        "CustId", "Name", "EmailId", "Region", "OrderId",
        "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount"
    )
    .withColumn("IsActive", F.lit(True))
    .withColumn("StartDate", current_timestamp)
    .withColumn("EndDate", F.lit(None).cast("timestamp"))
    )

    return new_records
except:
    # First run - create the table with all records as active
    return (
        dlt.read("customer_silver")
        .join(
            dlt.read("order_silver"),
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount"
        )
        .withColumn("IsActive", F.lit(True))
        .withColumn("StartDate", current_timestamp)
        .withColumn("EndDate", F.lit(None).cast("timestamp"))
    )

```

Step 6: Create Customer Aggregate Spend Table

This step creates the `customeraggregatespend` table, which aggregates customer spending by name and date.

```

@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date",
    table_properties={
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    },
    partition_cols=["Date"]
)
def customeraggregatespend():
    return (
        dlt.read("ordersummary")
        .filter(F.col("IsActive") == True)
        .groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))
    )

```